

UART, SPI & I2C — Deep Dive

Day 3

Day 3 — Embedded Systems 7-Day Study Series



7-Day Course Roadmap

Day	Topic
Day 1 ✓	Introduction — Architecture, GPIO, Memory, Protocols
Day 2 ✓	Interrupts, Timers & PWM
Day 3 →	UART, SPI, I2C — Deep Dive
Day 4	ADC/DAC & Sensor Interfacing
Day 5	RTOS Fundamentals (FreeRTOS)
Day 6	Power Management & Low-Power Design
Day 7	Bootloaders, OTA & System Design Project

1. UART — Universal Asynchronous Receiver/Transmitter

UART is the simplest serial protocol — **asynchronous** (no shared clock), point-to-point, full-duplex. Data is framed with start/stop bits and optionally a parity bit. Both sides must agree on **baud rate** before communication.

UART Frame Structure:

Field	Bits	Value	Purpose
Start bit	1	Always 0	Signals beginning of frame
Data bits	5–9	LSB first	Payload (usually 8 bits)
Parity bit	0–1	Even/Odd/None	Error detection
Stop bit(s)	1–2	Always 1	Line returns to idle (high)

Common Baud Rates & Error Tolerance:

Baud Rate	Bits/sec	Max Cable Length	Typical Use
9600	9,600	~15 m	Debug console, sensors
115200	115,200	~3 m	General purpose, GPS
460800	460,800	~1 m	Bluetooth modules
1M+	1,000,000+	<1 m	High-speed logging

Key Configuration Points:

- **Baud rate mismatch** — even 2% error causes frame corruption; use crystal oscillator, not RC clock.
- **RS-232 vs TTL UART** — RS-232 uses ± 12 V logic; use MAX232 level-shifter for MCU interfacing.

- **RS-485** — differential pair, multi-drop bus (up to 32 nodes), 1200 m range at low baud.
- **Flow control** — RTS/CTS hardware or XON/XOFF software; needed at high baud with slow processors.
- **DMA-driven RX** — use idle-line interrupt + DMA for robust variable-length packet reception.

HAL UART Transmit / Receive (STM32):

```
// Blocking TX – send 13 bytes, 100 ms timeout
HAL_UART_Transmit(&huart2;, (uint8_t*)"Hello World!\n", 13, 100);

// Non-blocking RX with interrupt – receive 1 byte
HAL_UART_Receive_IT(&huart2;, ■_byte, 1);

// DMA RX callback (fires when buffer full)
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *h) {
    process_packet(rx_buf, BUF_LEN);
    HAL_UART_Receive_DMA(h, rx_buf, BUF_LEN);
}
```

PRO TIP

For reliable UART RX, combine DMA circular mode with the IDLE line interrupt (UART_IT_IDLE). This lets you detect end-of-packet without knowing its length in advance.

■ **2. SPI — Serial Peripheral Interface**

SPI is a **synchronous**, full-duplex, master-slave protocol using 4 wires: **SCLK** (clock), **MOSI** (master→slave), **MISO** (slave→master), **CS/NSS** (chip select, active low). It achieves speeds of 10–100+ MHz, making it ideal for displays, flash memory, and ADCs.

SPI Clock Modes (CPOL & CPHA):

Mode	CPOL	CPHA	Clock Idle	Data Sampled On	Common Devices
0	0	0	Low	Rising edge	SD card, most sensors
1	0	1	Low	Falling edge	Some ADCs
2	1	0	High	Falling edge	Some DACs
3	1	1	High	Rising edge	ST7735 display

Multi-Slave Wiring Strategies:

Strategy	Wires	Pros	Cons
Individual CS lines	4+N	Simple, full speed each	N GPIO pins needed
Daisy-chain	4	Single CS, few pins	All slaves active; slow
Decode via 74HC138	4+3	Up to 8 slaves, 3 pins	Extra IC required

HAL SPI Full-Duplex Transfer:

```
// Pull CS low to select device
HAL_GPIO_WritePin(CS_GPIO_Port, CS_Pin, GPIO_PIN_RESET);

// Simultaneous TX+RX – 4 bytes, 10 ms timeout
HAL_SPI_TransmitReceive(&hspi1;, tx_buf, rx_buf, 4, 10);

// Release CS – deselect device
HAL_GPIO_WritePin(CS_GPIO_Port, CS_Pin, GPIO_PIN_SET);
```

SPI Best Practices:

- Always check the device datasheet for correct CPOL/CPHA mode — wrong mode = garbled data.
- Add 10–100 ns CS setup/hold time after toggling — use NOP instructions or GPIO delay.
- Use DMA SPI for large transfers (displays, SD cards) to free CPU during bulk data moves.
- Keep SPI traces short and impedance-matched at high speeds; avoid long breadboard wires.

IMPORTANT

SPI has no built-in ACK mechanism. If you need to verify receipt, implement application-layer acknowledgement or use a status register read after each command.

■ 3. I2C — Inter-Integrated Circuit

I2C uses just **2 wires** — **SDA** (data) and **SCL** (clock) — both open-drain with pull-up resistors. Multiple masters and up to 127 devices share the same bus, each addressed by a unique 7-bit (or 10-bit) address. Speed modes: Standard (100 kHz), Fast (400 kHz), Fast+ (1 MHz), High-Speed (3.4 MHz).

I2C Transaction Anatomy:

Step	Bus State	Description
1	START condition	SDA falls while SCL is high — master takes control
2	7-bit address + R/W	Master broadcasts target address + read(1)/write(0) bit
3	ACK from slave	Addressed slave pulls SDA low for 1 clock = acknowledged
4	Data byte(s)	8 data bits transferred, MSB first
5	ACK / NACK	Receiver ACKs each byte; NACK signals end or error
6	STOP condition	SDA rises while SCL is high — bus released

Pull-up Resistor Selection:

Speed Mode	Frequency	Recommended Pull-up	Max Bus Capacitance
Standard	100 kHz	4.7 kΩ	400 pF
Fast Mode	400 kHz	2.2 kΩ	400 pF
Fast Mode+	1 MHz	1 kΩ	550 pF

HAL I2C Register Read (e.g. MPU-6050 IMU):

```
#define MPU6050_ADDR 0x68 << 1 // HAL uses 8-bit address
#define WHO_AM_I_REG 0x75

// Write register address, then read value
uint8_t reg = WHO_AM_I_REG, val;
HAL_I2C_Master_Transmit(&hi2c1, MPU6050_ADDR, &reg, 1, 10);
HAL_I2C_Master_Receive (&hi2c1, MPU6050_ADDR|1, &val, 1, 10);

// Or use the convenience function:
HAL_I2C_Mem_Read(&hi2c1, MPU6050_ADDR, WHO_AM_I_REG,
                I2C_MEMADD_SIZE_8BIT, &val, 1, 10);
```

Common I2C Pitfalls:

- **Bus stuck LOW** — slave holds SDA low after power loss; fix by toggling SCL 9 times or power cycling.
- **Address conflict** — two devices share same address; use address pins (A0/A1/A2) to differentiate.
- **Missing pull-ups** — bus floats; never forget external pull-up resistors (internal MCU pull-ups too weak).
- **Clock stretching** — slow slaves hold SCL low; ensure master timeout is long enough.
- **HAL address shift** — STM32 HAL expects left-shifted 8-bit address; multiply 7-bit address by 2.

DEBUG TIP

Use a logic analyser (e.g. Saleae, cheap 8-ch clone) to debug I2C/SPI/UART issues. Pulseview + sigrok is a free open-source alternative that works with many cheap analysers.

4. Protocol Comparison

Feature	UART	SPI	I2C
Wires	2 (TX, RX)	4 (+ 1/slave)	2 (shared bus)
Speed	Up to ~5 Mbps	10–100+ MHz	100k–3.4 MHz
Topology	Point-to-point	Master + N slaves	Multi-master bus
Duplex	Full duplex	Full duplex	Half duplex
Clock	Async (baud)	Sync (SCLK)	Sync (SCL)
Addressing	None	CS line per slave	7/10-bit address
ACK/NAK	None (SW only)	None	Hardware ACK
Max distance	~15 m (RS-232)	<1 m (PCB)	~1 m (PCB)
Complexity	Low	Medium	Medium–High
Best for	Debug, GPS, BT	Flash, display	Sensors, EEPROM

5. DMA & LL Drivers Deep Dive

DMA (Direct Memory Access) allows peripherals to transfer data directly to/from RAM without CPU involvement. **LL (Low-Layer)** drivers give register-level control with minimal HAL overhead, critical for timing-sensitive or resource-constrained applications.

DMA Mode	Use Case	Trigger
Normal	One-shot transfer (e.g. TX buf)	Software / peripheral request
Circular	Continuous ADC / UART RX	Auto-restarts on completion
Mem-to-Mem	memcpy offload	Software trigger only
Double-buffer	Ping-pong audio/streaming	Switches buffer on half-complete

HAL vs LL vs Register — Speed Hierarchy:

HAL_UART_Transmit() → ~10–50 µs overhead per call (safe, portable)
LL_USART_TransmitData8() → ~1–2 µs (thin wrapper, no state machine)
Direct register: USART1->DR = byte → <100 ns (bare metal, no checks)

RULE OF THUMB

Use HAL for prototyping and portability. Switch to LL or direct register access only for proven bottlenecks. Profile first with DWT cycle counter before optimising.

6. Practice Exercises

#	Exercise	Diff	Time
1	UART echo: receive bytes over USART2, echo back uppercase via interrupt	■■	30 min
2	Read BME280 temperature/humidity via I2C; print to UART terminal	■■■	60 min

3	Write/read 256 bytes to SPI NOR flash (W25Q32); verify CRC match	■■■	60 min
4	DMA UART RX: receive variable-length packet using IDLE-line interrupt	■■■■	90 min
5	I2C scanner: detect all devices on bus, print addresses via UART	■■	30 min
6	SPI display: render text + rectangle on ST7735 TFT using DMA SPI	■■■■	90 min

7. Quick-Reference Cheat Sheet

Concept / API	Key Point
<code>HAL_UART_Transmit_DMA()</code>	Non-blocking TX; fires <code>TxCpltCallback</code> when done
<code>HAL_UART_Receive_IT()</code>	Enable RXNE interrupt; restart in <code>RxCpltCallback</code>
UART IDLE interrupt	Best way to detect end of unknown-length packet
<code>HAL_SPI_TransmitReceive()</code>	Full-duplex; TX and RX happen simultaneously
SPI NSS pin	Drive manually (GPIO) for multi-slave; HW NSS unreliable
<code>HAL_I2C_Mem_Read/Write()</code>	Convenience: sends reg addr + data in one transaction
I2C address (HAL)	Always left-shift 7-bit addr: <code>addr << 1</code>
DMA circular mode	Buffer auto-reloads; use half/full callbacks for double-buf
<code>LL_USART_IsActiveFlag_RXNE()</code>	Poll RX ready flag — fastest polling approach
Logic level mismatch	3.3V MCU + 5V device → use level shifter or voltage divider

Day 3 Summary & Next Steps

■ Covered Today — Day 3	➔ Coming Up — Day 4
• UART frame structure & baud rate config • RS-232 / RS-485 variants & level shifting • SPI CPOL/CPHA modes & CS management • I2C addressing, ACK/NACK, pull-ups • DMA modes: normal, circular, double-buffer • HAL vs LL vs register-level trade-offs	• ADC resolution, sampling rate, oversampling • DAC output, R-2R ladder alternative • Sensor interfacing: NTC, hall effect, strain • Calibration techniques and offset correction • Filtering: moving average, IIR, Kalman intro • DMA-triggered ADC continuous conversion

Study tip: Use STM32CubeIDE's built-in serial monitor for UART debugging. A cheap logic analyser (8-ch, ~\$10) is invaluable for visualising SPI/I2C transactions.